Laboratorio

Corso di Basi di Dati e Web

A.A. 2023-2024

Progetto “Gestione aziende ospedaliere”

Autore:

Angelica Porcelli angelica.porcelli@studenti.unimi.it

Matricola: 24140A

1. Progettazione concettuale

Il seguente schema concettuale include le seguenti entità principali (Paziente, Ricovero, Prenotazione, Esame, Personale, Ospedale, Reparto, Ambulatorio, Pronto Soccorso, Turni) e le loro relazioni.

Per quanto riguarda l'insieme di ospedali, sono identificati da un codice univoco e al momento della creazione del database, assumo che vengano creato senza nulla, quindi le cardinalità minime verso le sue relazioni sono zero. Un ospedale può disporre di più ambulatori e ogni ambulatorio può riferirsi a vari ospedali, poiché assumo che per quanto riguarda gli ambulatori esterni, potrebbero far riferimento a più ospedali. La gestione degli ambulatori l'ho fatta tramite una generalizzazione, che ha come entità specializzate, l'ambulatorio interno e quello esterno. Questa è una generalizzazione totale e sovrapposta, poiché un ambulatorio può essere solamente di due tipi interno o esterno e c'è anche la possibilità che un ambulatorio sia per alcuni ospedali esterno e per solo uno interno. Assumo che un ospedale può avere al massimo un pronto soccorso, ma potrebbe anche non averne, quindi l'ospedale partecipa con cardinalità opzionale e singola verso il PS. Nel caso in cui l'ospedale avesse un pronto soccorso, si gestiscono i turni del personale sanitario, ovvero degli infermieri e dei medici, questi sono rappresentati tramite una generalizzazione, con turno infermiere e medico come figli, con una relazione con le due entità di riferimento. È una generalizzazione totale sovrapposta, poiché un turno potrebbe riguardare sia un medico che un infermiere e fa riferimento solo a questo tipo di personale ospedaliero.

Il personale dell'ospedale, viene rappresentato tramite una generalizzazione parziale ed esclusiva, poiché assumo che una persona che lavori nel reparto di un ospedale, possa essere di tipi diversi, rispetto ai tre specificati, in tal caso rimane parte del personale generico, invece è esclusivo poiché una persona non può fare riferimento a più di una tipologia di personale.

Un medico potrebbe aver ricevuto una promozione ed essere diventato un vice primario oppure un primario, ma non per forza uno dei due, quindi lo sviluppo tramite una generalizzazione parziale ed esclusiva, poiché nel momento in cui diventa primario, è perché ha preso una promozione e allora non sarebbe più vice primario.

Un primario può avere più di una specializzazione, quindi avrà un attributo multi-valore. Si vuole anche gestire la sostituzione di un primario e del vice-primario, quindi ho una relazione tra queste due entità, con le informazioni di data di inizio e data di fine della sostituzione.

Per rappresentare da cosa è formato un ospedale, ho utilizzato dei pattern di design comuni, ovvero *part of* per quanto riguarda i reparti e le stanze. Infatti, all'interno dell'ospedale possono esserci dei reparti, quindi l'ospedale concorre alla relazione con questa entità in modo opzionale e multiplo, poiché alla creazione non ha reparti e poi generalmente ne ha di diversi.

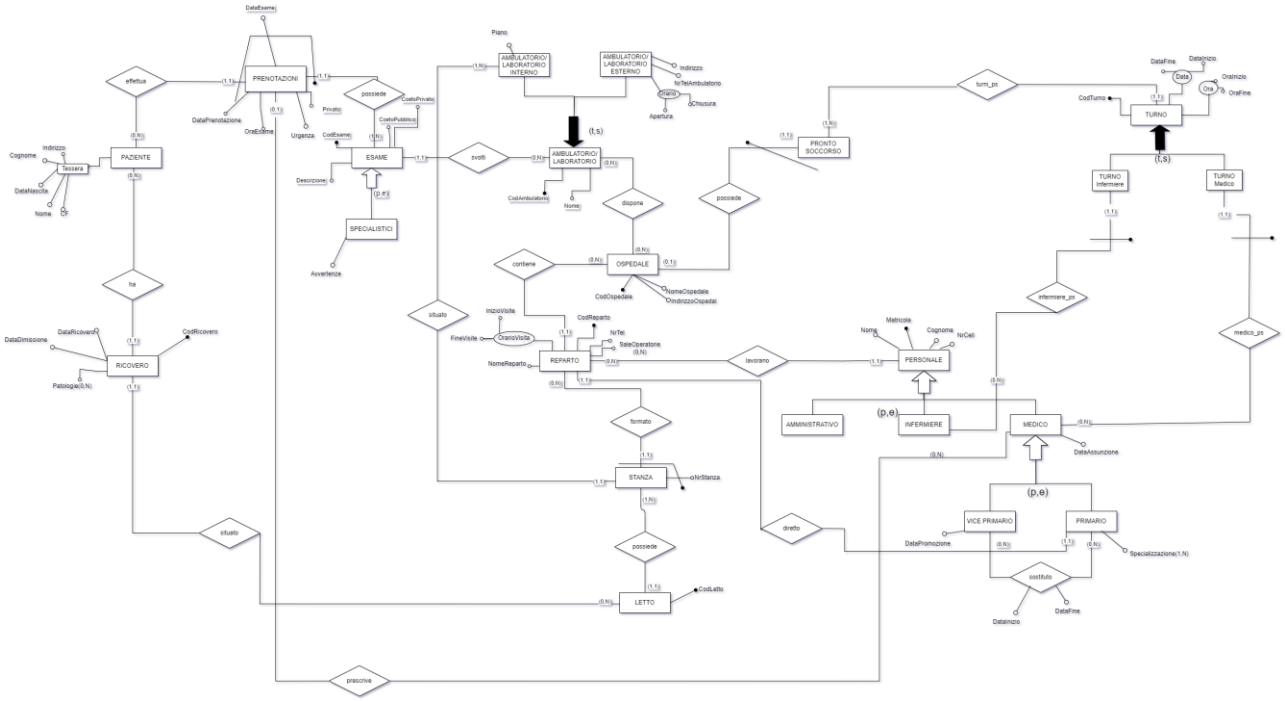
All'interno dei reparti lavorano persone del personale ospedaliero; in più un reparto può avere delle sale operatorie, quindi gestisco questa informazione con un attributo multi-valore. Un reparto può avere al massimo un primario, ma ha più vice primari, quindi per gestire il vincolo di unicità, creo una relazione unaria con il primario medico.

Un reparto ha una relazione con l'entità stanza molti a uno, con opzionalità da parte del reparto, poiché al momento della creazione potrebbe non avere delle stanze, oppure non

permettere il ricovero, perciò non avere delle stanze o averne diverse. Una stanza è identificata dal suo numero e dal reparto in cui si trova.
L'ambulatorio interno ha una relazione con l'entità stanza, poiché è situato all'interno di una specifica. Nella stanza sono presenti dei letti, con un codice identificativo, quindi la relazione tra stanza e letti è una molti a 1, poiché un letto specifico si trova solo all'interno di una stanza. Con l'entità letti, c'è una relazione con il ricovero, poiché un paziente se è ricoverato, si trova per forza in un letto, quindi è un'associazione uno a molti, con obbligatorietà da parte dell'entità ricovero ma opzionalità da parte del letto.
Per quanto riguarda la gestione della prenotazione degli esami, l'entità paziente, viene identificata dalla propria tessera sanitaria, formata quindi da vari attributi, questa informazione la modello tramite un attributo composto. L'entità paziente ha 2 relazioni, per quanto riguarda la prenotazione ha cardinalità minima zero, poiché è opzionale, mentre quella massima è N, infatti un paziente può effettuare varie prenotazioni, purché non svolga uno stesso esame in uno stesso giorno. Una prenotazione viene identificata univocamente dal paziente che sta prenotando, dalla data dell'esame e dall'esame da svolgere, perciò le relazioni di questa istanza sono con cardinalità minima obbligatoria e quella massima è uno. L'entità esame che si relaziona con la prenotazione fa riferimento a un insieme di esami generici, con la possibilità di essere specialistico, quindi lo rappresento tramite una generalizzazione, parziale ed esclusiva, poiché potrebbe essere specialistico come non. Per gli esami specialistici bisogna recuperare l'informazione del medico che li ha prescritti, quindi nella prenotazione abbiamo una relazione con il medico, che è opzionale perché se fossero esami non specialistici, non sarebbe necessario avere un medico che li prescriva, mentre un medico, potrebbe prescrivere vari esami, quindi è uno a molti.
Un paziente, potrebbe essere ricoverato, in tal caso quindi fa riferimento al ricovero con un codice identificativo. Potrebbe essere ricoverato con zero o più patologie, che potrebbero essere diverse, quindi utilizzo un attributo multi-valore per renderlo possibile. Una persona ricoverata, significa che si trova all'interno di un letto, quindi questa entità ha una relazione uno a molti con l'entità letto, poiché una persona nel momento del ricovero si può trovare in un solo letto e il letto può aver avuto diversi pazienti, in momenti diversi.

1.1 Schema ER

https://www.drawio.com/

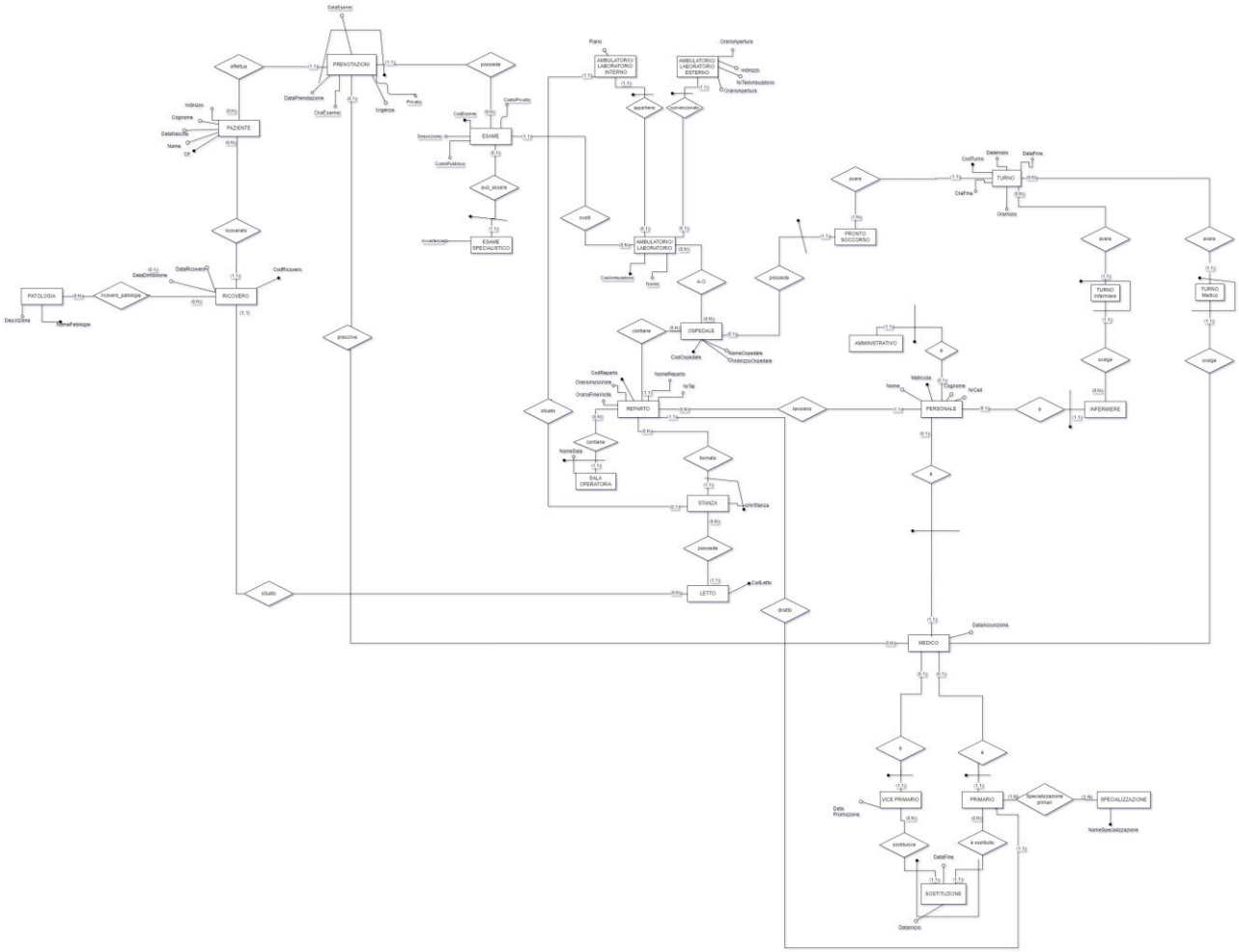


1.2 Vincoli di dominio

1. Un Esame ha un urgenza, che lo descrive. Le urgenze sono in ordine di gravità: verde, giallo, rosso.
2. Un Esame ha specificati 2 costi, uno in regime privato e uno in pubblico, nel momento in cui un paziente lo prenota, avrà a carico uno solo dei due costi.
3. Un Reparto ha un solo primario di riferimento, mentre i vice-primari possono essere diversi, con la possibilità di sostituirli.
4. Un Primario ha la data di promozione che deve essere successiva alla data di assunzione.
5. Un Paziente può fare una prenotazione per un esame in una data specifica
6. Un paziente può essere ricoverato con un caso alla volta, se ha di riferimento due ricoveri allora deve averli distanziati nel tempo, con uno con data di dimissione passata.
7. Il numero di telefono di un reparto deve essere unico.
8. Un Letto può essere occupato solo se è libero, ovvero il Ricovero deve avere una data di dimissione passata.
9. Un Esame, se fosse specialistico dovrebbe avere un medico che l'ha prescritto.

2. Progettazione logica

2.1 Schema ER ristrutturato



2.2 Vincoli di dominio

1. Per un Reparto, l'orario di fine visita, deve essere successivo all'orario di inizio visita.
2. Nel Turno, la data di fine deve essere uguale o successiva alla data di inizio.
3. Nel Turno, l'ora di fine deve essere successiva all'ora di inizio, a meno che la data di fine sia successiva alla data di inizio.
4. Nella Sostituzione di un Primario con un Vice, la data di inizio deve essere precedente a quella di fine. La data di fine potrebbe anche essere nulla.

2.3 Modello relazionale

Legenda:

ammette valore nullo unicità dell'attributo chiave primaria *chiave esterna*

Entità forti:

OSPEDALE(CodOspedale, NomeOspedale, IndirizzoOspedale)

REPARTO(CodReparto , NomeReparto, NrTelefono, OrarioInizioVisite, OrarioFineVisite, , *CodOspedale*)

Foreign key (CodOspedale) references Ospedale(CodOspedale)

PERSONALE(Matricola, Nome, Cognome, NrCell, *CodReparto*)

Foreign key (CodReparto) references Reparto(CodReparto)

SPECIALIZZAZIONE(NomeSpecializzazione)

TURNO(CodTurno, DataOraInizio, DataOraFine, *CodOspedalePS*)

Foreing key(CodOspedalePS) references ProntoSoccorso(CodOspedale)

LETTO(CodLetto, Nr*Stanza, CodReparto*)

Foreign key(NrStanza, CodReparto) references Stanza(NrStanza, CodReparto)

PAZIENTE(CF, Nome, Cognome, DataDiNascita, Indirizzo)

ESAME(CodEsame, Descrizione, CostoPubblico, CostoPrivato, *CodAmbulatorio*)

Foreign key (CodAmbulatorio) references Ambulatorio(CodAmbulatorio)

RICOVERO(CodRicovero, DataRicovero, DataDimissione, *CFPaziente*, *CodLetto*)

Foreign key (CFPaziente) references Paziente(CF)

Foreign key (CodLetto) references Letto(CodLetto)

PATOLOGIA(NomePatologia, Descrizione)

AMBULATORIO_LABORATORIO (CodAmbulatorio, Nome)

Entità deboli:

MEDICO(*MatricolaMedico,* DataAssunzione)

Foreign key (MatricolaMedico) references Personale(Matricola)

VICE PRIMARIO(*MatricolaVicePrimario*, DataPromozione)

Foreign key (MatricolaVicePrimario) references Medico(Medico)

PRIMARIO(*MatricolaPrimario, CodReparto*)

Foreign key (MatricolaPrimario) references Medico(MatricolaMedico)

Foreign key (CodReparto) references Reparto(CodReparto)

SOSTITUIZIONE(*MatricolaPrimario, MatricolaViceprimario,* DataInizio, DataFine)

Foreign key(MatricolaPrimario) references Primario(MatricolaPrimario)

Foreign key(MatricolaVicePrimario) references VICEPRIMARIO(MatricolaVicePrimario)

SPECIALIZZAZIONE_PRIMARI(*MatricolaPrimario, Specializzazione*)

Foreign key(MatricolaPrimario) references Primario(MatricolaPrimario)

Foreign key(Specializzazione) references Specializzazione(NomeSpecializzazione)

INFERMIERE(*MatricolaInfermiere*)

Foreign key (MatricolaInfermiere) references Personale(Matricola)

AMMINISTRATIVO(*MatricolaAmministrativo*)

Foreign key (MatricolaAmministrativo) references Personale(Matricola)

PRONTO SOCCORSO(*CodOspedale*)

Foreign key (CodOspedale) references Ospedale(CodOspedale)

TURNO_INFERMIERE(*CodTurno, MatricolaInfermiere)*

Foreign key(CodTurno) references Turno(CodTurno)

Foreign key(MatricolaInfermiere) references Infermiere(Infermiere)

TURNO_MEDICO(*CodTurno, MatricolaMedico*)

Foreign key(CodTurno) references Turno(CodTurno)

Foreign key(MatricolaMedico) references Medico (MatricolaMedico)

SALA_OPERATORIA(*CodReparto,* NomeSala)

Foreign key(CodReparto) references Reparto(CodReparto)

STANZA(NrStanza, *CodReparto*)

Foreign Key(IdReparto) references reparto(CodReparto)

PRENOTAZIONE(OraEsame, DataPrenotazione, Privato, Urgenza, DataEsame, *CFPaziente, Esame, MatricolaMedico*)

Foreign key (CFPaziente) references Paziente(CF)

Foreign key (CodEsame) references Esame (CodEsame)

Foreign key (MatricolaMedico) references Medico(MatricolaMedico)

ESAME SPECIALISTICO(*EsameSpecialistico*, Avvertenze)

Foreign key(EsameSpecialistico) references Esame(CodEsame)

RICOVERO_PATOLOGIE*(CodRicovero, NomePatologia*)

Foreign key(Patologia) references Patologia(NomePatologia)

Foreign key(CodRicovero) references Ricovero(CodRicovero)

AMBULATORI_OSPEDALI(*CodAmbulatorio, CodOspedale*)

Foreign key(CodAmbulatorio) references Ambulatorio_Laboratorio(CodAmbulatorio)

Foreign key(CodOspedale) references Ospedale(CodOspedale)

AMBULATORIO_LABORATORIO_INTERNO(*CodAmbulatorio,* Piano, *NrStanza, CodReparto)*

Foreign key(NrStanza, CodReparto) references Stanza(NrStanza, CodReparto)

Foreign key(CodAmbulatorio) references Ambulatorio_ Laboratorio(CodAmbulatorio)

AMBULATORIO_LABORATORIO_ESTERNO(*CodAmbulatorio*, Indirizzo, NrTelAmbulatorio, OrarioApertura, OrarioChiusura)

Foreign key(CodAmbulatorio) references Ambulatorio_Laboratorio (CodAmbulatorio)

2.4 Codice SQL

Creazione del database: Backup_Schema.sql SchemaOspedale.sql
Inserimento dei dati nel database: Backup_Dati.sql DatiOspedale.sql

Interrogazioni richieste:
a. Determinare i vice primari che hanno sostituito esattamente una volta il proprio primario.

```
SELECT MatricolaVicePrimario
FROM SOSTITUZIONE
GROUP BY MatricolaVicePrimario
HAVING COUNT(*) = 1
```

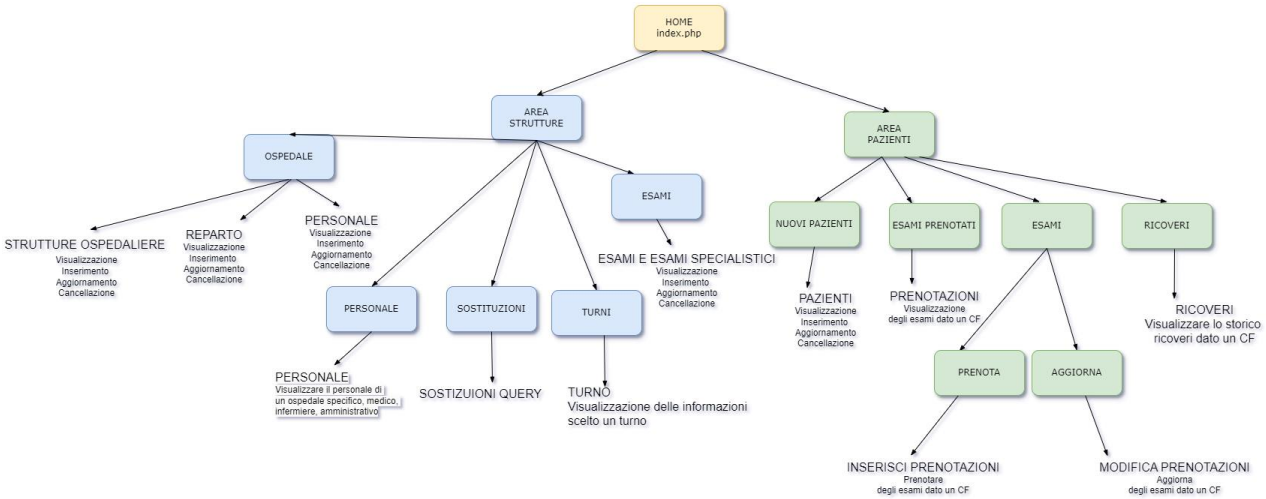
b. Determinare i vice primari che hanno sostituito almeno due volte il proprio primario

```
SELECT MatricolaVicePrimario
FROM SOSTITUZIONE
GROUP BY MatricolaVicePrimario
HAVING COUNT(*) > 1
```

c. Determinare i vice primari che non hanno mai sostituito il proprio primario

```
SELECT MatricolaVicePrimario
FROM VICE_PRIMARIO
EXCEPT
SELECT MatricolaVicePrimario
FROM SOSTITUZIONE
```

3. Progettazione del sito



Nome del database PostgreSQL: Ospedale
Nome della cartella sotto htdocs: ProgettoOspedale
http://localhost/ProgettoOspedale/index.php